Guide d'Implémentation - Service de Reconnaissance Faciale de reconnaissance faciale

Vue d'ensemble

Ce document décrit l'architecture et les endpoints nécessaires pour implémenter un service de reconnaissance faciale biométrique.

Objectif de cette implémentation

1. Démarrer une caméra
2. Diffuser un flux vidéo temps réel (MJPEG)
3. Récupérer des métriques biométriques en temps réel (liveness, estimation de l'âge, du genre...)
4. Capturer le visage avec un recadrage (seule la portion correspondant au visage est récupérée)
5. Vérifier que cette image respecte un ensemble de référentiels biométriques
6. Soumettre cette image au moteur de reconnaissance ABIS
7. Retourner les informations démographiques associées

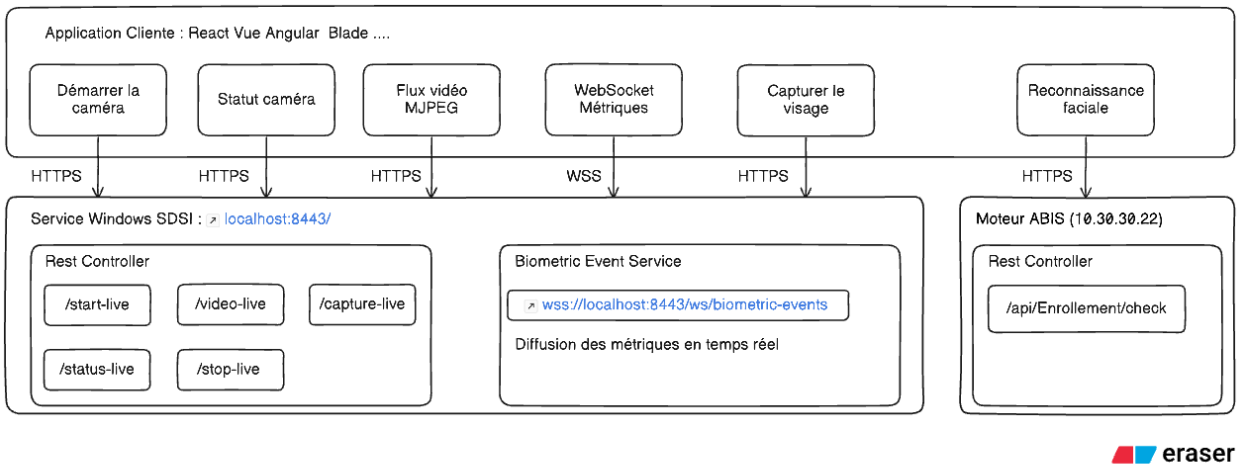
1. Client → /start-live
   └─> Service démarre 2 threads concurrents :
   - Thread OpenCV : Capture webcam → Encode JPEG
   - Thread Metrics: Analyse frames → Calcul métriques

2. Client se connecte au flux :
   - GET /video-live → Stream MJPEG pour affichage
   - WSS /ws/biometric-events → Métriques temps réel

3. Client demande capture :
   - GET /capture-live → Image base64 + métriques

4. Client envoie AU MOTEUR ABIS :
   - POST https://10.30.30.22/api/Enrollement/check
   - Reçoit les données démographiques
   - Affichage des données démographiques

5. Client arrête :
   - GET /stop-live → Libération ressources



Vue d'ensemble de toutes les intégrations à effectuer pour la reconnaissance faciale

Endpoints à Implémenter

Service de Streaming Vidéo (https://localhost:8443)

Démarrer la Caméra : GET /start-live

Objectif de cet endpoint :

- Initialiser la capture vidéo depuis la caméra
- Démarrer le traitement des frames
- Retourner le statut d'initialisation

```
const startCamera = async () => {
  // 1. Appeler du service pour démarrer la caméra
  const startResponse = await fetch(`https://localhost:8443/start-live`);
  const startData = await startResponse.json();

  if (startResponse.ok && startData.status === 'started') {
    setFaceConnectionStatus('connected');

    // 2. Démarrer le flux vidéo MJPEG
    startVideoStream();
  }
};
```

```
{
  "status": "started",
  "message": "Camera started successfully"
}
```

```
{
  "status": "error",
  "message": "Camera unavailable"
}
```

Flux Vidéo MJPEG : GET /video-live

Objectif de cet endpoint :

- Stream continu d'images JPEG
- Résolution : 1280×720

```
const startVideoStream = () => {
    // URL du flux MJPEG (serveur backend)
    const videoUrl = `https://localhost:8443/video-live`;

    if (videoImgRef.current) {
        // Assigner l'URL au src de l'image
        videoImgRef.current.src = videoUrl;

        // Callbacks pour gérer le chargement
        videoImgRef.current.onload = () => {
          addFaceLog('Flux vidéo actif');
        };

        videoImgRef.current.onerror = () => {
            addFaceLog('Erreur flux vidéo');
            setFaceConnectionStatus('error');
        };
    }
};
```

Une fois `videoImgRef.current.src` défini sur l'URL MJPEG : ***https://localhost:8443/video-live***, le navigateur **reçoit automatiquement** un flux continu d'images JPEG.

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace; boundary=frame

--frame
Content-Type: image/jpeg

[JPEG binary data de Frame 1]
--frame
Content-Type: image/jpeg

[JPEG binary data de Frame 2]
--frame
Content-Type: image/jpeg

[JPEG binary data de Frame 3]
--frame
...
```

Puis ce dernier affiche automatiquement chaque nouvelle frame dans l'élément `<img>`, créant l'**illusion d'une vidéo**.

```
<h4>Regarder la caméra</h4>
<img
   ref={videoImgRef}
  alt="Flux vidéo de la caméra"
  style={{ display: faceConnectionStatus === 'connected' ? 'block' : 'none' }
 />
```

Statut de la Caméra : GET /status-live

Objectif de cet endpoint :

- Vérifier si la caméra est active
- Retourner le nombre de frame du flux vidéos

```
{
  "running": true,
  "frameCount": 1523,
}
```

Capturer une Image : GET /capture-live

Objectif de cet endpoint :

- Capturer la frame courante
- Effectuer la détection de visage
- Calculer les métriques biométriques
- Encoder l'image en base64
- Valider la qualité (netteté, luminosité, liveness)

```
{
      "image": "image base64 ",
   "metrics": {
      "brightness": -7661,
      "rightEyeStatus": 2988,
      "gender": -2935.0972,
      "contrast": -5955,
      "leftEyeStatus": 4091,
      "glassStatus": -6535,
      "sharpness": -8695,
      "mouthStatus": 1916,
      "fastPassiveLiveness": 97.712234,
      "age": 25.695026
   },
```

```
  "status": "success"
}
```

```
{
  "message": "Aucun visage détecté"
}
```

```
{
  "message": "2 visages détectés (plusieurs personnes)"
}
```

NB : Vérification post-capture

Comme mentionné lors de nos dernières présentations de la solution, nous laissons l'agent capturer le visage même en présence de LED rouges. Mais une fois la capture effectuée, vous obtiendrez une image en base64 ainsi que les métriques associées à cette image (cette frame à l'instant T).

Il vous faudra ensuite traiter ces métriques et, tant que les conditions que vous aurez définies ne sont pas respectées, l'agent devra reprendre la capture de l'image. Une fois que toutes les métriques auront passé les filtres que vous aurez fixés (par exemple : pas de lunettes, bouche fermée, yeux ouverts, etc.), vous pourrez passer à l'étape suivante.

Le but de cette vérification post-capture est de s'assurer de recevoir des images de bonne qualité qui seront soit stockées, soit traitées par le moteur ABIS.

Envoi de la capture au moteur ABIS pour la reconnaissance faciale -POST : /api/Enrollement/check.

Une fois le visage capturé avec de bonnes métriques attestant de sa qualité, l'étape suivante consiste à envoyer cette image à notre moteur ABIS pour la reconnaissance faciale, accompagnée des informations démographiques de la personne identifiée.

Pour cela vous devrez mettre le visage capturer dans un FormData, puis l'envoyer via une requette HTTPS POST : /api/Enrollement/check.

```
// Envoi de la capture au backend pour la vérification
const sendBiometricData = async (faceImage) => {
  try {
    const formData = new FormData();
    const response = await fetch(faceImage);
    const blob = await response.blob();
    formData.append("request", blob, "Face.png");
    console.log([...formData.entries()]);

    const res = await fetch(`https://10.30.30.22/api/Enrollement/check`, {
      method: "POST",
      body: formData,
    });

    if (!res.ok) {
      const text = await res.text();
      throw new Error(`Erreur serveur ${res.status}: ${text}`);
    }

    const data = await res.json();
    console.log("Réponse backend:", data);

    // Données démographiques associées à la personne identifiée et envoyé
es par le moteur ABIS
    setPerson({
      idPersonne: data.idPersonne,
      nom: data.nom,
```

```
            prenom: data.prenom,
            dateNaiss: data.dateNaiss,
            numeroBeneficiaire: data.numeroBeneficiaire,
            numeroAssure: data.numeroAssure,
            sexe: data.sexe,
            regime: data.regime,
            ftpFace: data.ftpFace,
            hasRight: data.hasRight,
            beneficiaires: data.beneficiaire // tableau de personne
        });

    } catch (error) {
        console.error("Erreur lors de l'envoi des données biométriques:", error);
        throw error;
    }
};
```

Arrêter la Caméra : GET /stop-live

Objectif de cet endpoint :

- Arrêter la capture vidéo
- Libérer les ressources
- Nettoyer les buffers

```
{
    "type": "CAMERA_STOPPED",
    "message": "Caméra arrêtée - 2 threads stoppés",
    "data": null,
    "timestamp": "2025-12-20T15:07:53.1268613"
}
```

WebSocket - Métriques en Temps Réel

Connexion WebSocket : wss://localhost:8443/ws/biometric-events

Ce WebSocket aura pour unique but de transmettre les métriques en temps réel. Ces métriques ne sont disponibles que s'il y a une seule personne en face de la caméra. S'il n'y a personne ou s'il y a plusieurs personnes, les métriques ne seront pas récupérées. À la place, vous recevrez des messages tels que « Aucun visage détecté » ou « Deux visages détectés » pour vous en informer.

```
{
  "type": "QUALITY_CHECK",
  "message": "Aucun visage détecté",
  "data": null,
  "timestamp": "2025-12-20T15:13:59.8005295"
}
```

```
{
  "type": "QUALITY_CHECK",
  "message": "Deux visages détectés",
  "data": null,
  "timestamp": "2025-12-20T15:13:59.8005295"
}
```

```
{
  "type": "QUALITY_METRICS",
  "message": "Frame #6562 | Netteté: -7812.00⚠️ | Luminosité: -6647.00⚠️ |
Contraste: -5380.00⚠️ | Bouche: 1078.0 | Œil D: 2429.0 | Œil G: 2699.0 | Lune
```

```
ttes: -2788.0 | Liveness: 91.89 | Age: 25.77 | Gender: -4560.85 | Size: 119.63 |
Confidence: 2756.00",
    "data": null,
    "timestamp": "2025-12-20T15:14:40.8286436"
}
```

Ensuite, l'utilisation d'une expression régulière est nécessaire ici pour récupérer l'objet contenant les métriques par image. Les métriques se mettent à jour en fonction du positionnement du visage ainsi que de l'environnement lors de la capture vidéo.

```
const retriveMetrics = (message) => {
    let regex = /([^|:]+):\s*([-+]?\d+(?:\.\d+)?)/g;
    const result = {};

    for (const match of message.matchAll(regex)) {
        const key = match[1].trim();
        const value = parseFloat(match[2]);
        result[key] = value;
    }

    return result;
}
```

```
{
    "Netteté": -3399,
    "Luminosité": -4765,
    "Contraste": -4260,
    "Bouche": 1035,
    "Œil D": 501,
    "Œil G": 425,
```

```
    "Lunettes": -854,
    "Liveness": 89.11,
    "Age": 25.47,
    "Gender": -3027.12,
    "Size": 125.1,
    "Confidence": 2941
}
```

```
JS test.js > retriveMetrics
1  let message = "Frame #6562 | Netteté: -7812.00 ⚠ | Luminosité: -6647.00 ⚠ | Contraste: -5380.00 ⚠ | Bouche: 1078.0 | Œil D:
2  const retriveMetrics = (message) => {
3      let regex = /([^|:]+):\s*([-+]?\d+(?:\.\d+)?)/g;
4      const result = {};
5
6      for (const match of message.matchAll(regex)) {
7          const key = match[1].trim();
8          const value = parseFloat(match[2]);
9          result[key] = value;
10     }
11     return result;
12 }
13
14 console.log(retriveMetrics(message))
15
16

PROBLEMS   OUTPUT   DEBUG CONSOLE   PORTS   POLYGLOT NOTEBOOK   NUGET   TERMINAL                                   bash

mugen@DESKTOP-MP1141L MINGW64 ~/Desktop/work/sdsi/carte cnss/code source/SCOSTA-SM (master)
$ node test.js
test expression reguliere
{
  'Netteté': -7812,
  'Luminosité': -6647,
  Contraste: -5380,
  Bouche: 1078,
  'Œil D': 2429,
  'Œil G': 2699,
  Lunettes: -2788,
  Liveness: 91.89,
  Age: 25.77,
  Gender: -4560.85,
  Size: 119.63,
  Confidence: 2756
}
```

Démonstration de la récupération des métriques sous le format d'un objet JavaScript exploitable

Netteté: -10000 à 10000 (>5000 = bon)
Luminosité: 0-100 (50-80 = idéal)
Contraste: 0-100 (>50 = bon)
Liveness: 0-100 (>85 = vivant)
Age: Âge estimé en années

```
Gender: <0 = masculin, >0 = féminin
Lunettes: <0 = sans lunettes, >0 = avec lunettes
Œil D/G: >0 = ouvert, <0 = fermé
Bouche: >0 = ouverte, <0 = fermée
```

Une fois l' objet récupéré, la prochaine étape consiste à interpréter ces données. Pour cela, nous vous avons fourni des intervalles par attribut dans la section ***Interprétation des métriques.***

```
<div className="mb-4">
  <div className="flex justify-between mb-1">
    <span className="text-sm font-medium">Liveness : </span>
  <span className="text-sm !font-medium">{ liveMetrics != null ? liveMetric
s?.Liveness > 85 ?  "🟩"  : "🟥" : "-"}</span>
  </div>
 </div>
```

Visage en cours de traitement	1
Clarté	-6493
Luminosité	-3907
Contraste :	-3201
Liveness :	🟩
Age :	24 ans
Genre :	♂️
Lunettes :	🟩
Oeil Droit :	🟩
Oeil Gauche :	🟩
Bouche :	🟩

Résultat final de l'affichage des métriques